COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 - Data Structures and Algorithms | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | August 5, 2026 | **DATE SUBMITTED:** | August 7, 2026 |
| **NAME:** | Libo-on, Gian Carlo G. | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**ACTIVITY NO. 5** **Queues**

## 1. PROCEDURE OUTPUT

**ILO A : QUEUE STL**

main.cpp

```cpp
#include<iostream>
#include<queue>


void displayAll(std::queue<char> copyQ);

int main(){


    //create an object
    std::queue<char> myQ;
    //use enqueue operations
    myQ.push('H');
    myQ.push('A');
    myQ.push('R');
    myQ.push('U');
    myQ.push('K');
    myQ.push('A');
    myQ.push('M');
    myQ.push('I');
    std::cout << "The current front is: " << myQ.front() << std::endl;
    std::cout << "The current back is: " << myQ.back() << std::endl;
    displayAll(myQ);
    //use dequeue operations

    myQ.pop();
    displayAll(myQ);
    myQ.pop();
    displayAll(myQ);
    myQ.pop();
```

```
    std::cout << "Is the queue empty? " << myQ.empty() << std::endl;
    return 0;
}

//note: only use the member funcions of the queue stl
        //create a copy of the queue
void displayAll(std::queue<char> copyQ){
        //lopp until empty
    while(!copyQ.empty()){
        //display the front
        std::cout << " " << copyQ.front() << " ";
        //dequeue the front
        copyQ.pop();
    }
//add new line
std::cout << std::endl;
}
```

Output

```
The current front is: H
The current back is: I
 H  A  R  U  K  A  M  I
 A  R  U  K  A  M  I
 R  U  K  A  M  I
Is the queue empty? 0
```

Observations

Implementing the queue by using the queue standard library allowed me to understand what commands does a queue have. It also helped me understand how each of the commands work and how it is almost similar to stacks if not for the queue being able to track the back end of the data. By following the FIFO rule, I am able to apply the most basic commands using the stl since it already handles everything for me. The commands are basically the same as stacks but with different implementations.

## ILO B : QUEUE and ARRAY LINKED LIST

main.cpp

```cpp
#include <iostream>
#include "queue_ll.h"


int main(){
    qNode<char>* front =  nullptr;
    qNode<char>* back = nullptr;

    enqueue('H', &front, &back);
    std::cout << "Front: " << front -> data << " Back: " << back -> data << std::endl;
    enqueue('A', &front, &back);
    std::cout << "Front: " << front -> data << " Back: " << back -> data << std::endl;
    enqueue('R', &front, &back);
    std::cout << "Front: " << front -> data << " Back: " << back -> data << std::endl;

    std::cout << "Testing the dequeue operator \n";
    dequeue(&front, &back);
    std::cout << "Front: " << front -> data << " Back: " << back -> data << std::endl;
    dequeue(&front, &back);
    std::cout << "Front: " << front -> data << " Back: " << back -> data << std::endl;


    std::cout << "Testing the display operator \n";
    display(front);
    enqueue('H', &front, &back);
    display(front);
    enqueue('A', &front, &back);
    (front);
    enqueue('R', &front, &back);
    display(front);

    std::cout << "Is Empty? " << (isEmpty(&front, &back) ? "Yes" : "No") << std::endl;
    return 0;

}
```

queue_ll.h

```h
#ifndef queue_ll_h
#define queue_ll_h
#include <iostream>

//create a class node
template <typename T>
class qNode{
    public:
        T data;
        qNode* next;
};
//new node creator
template<typename T>
qNode<T>* new_node(T newData){
    //allocate a space for the new node
    qNode<T>* newNode = new qNode<T>;

    //store the newdata to the newNode
    newNode -> data = newData;

    //point to newNode next to null
    newNode -> next = nullptr;

    return newNode;
}

//enqueue
template<typename T>
void enqueue(T newData, qNode<T>** frontptr, qNode<T>** backptr){

    //create a new node
    qNode<T>* newNode = new_node(newData);
    //check if the queue is empty
    if((*frontptr) == nullptr && (*backptr) == nullptr){
        (*frontptr) = newNode;
        (*backptr) = newNode;
        return;
    }
    //if not then insert at the back
    (*backptr) -> next = newNode;
    (*backptr) = newNode;
};
```

```cpp
//dequeue
template<typename T>
void dequeue(qNode<T>** frontptr, qNode<T>** backptr){
    //check if the queue is empty
    if((*frontptr) == nullptr && (*backptr) == nullptr){
        std::cout << "THE QUEUE IS EMPTY." << std::endl;
        return;
    }
    //create a temporary variable to store the node to be deleted
    qNode<T>* deleteTemp = nullptr;
    //assign the current front to the deleteTemp
    deleteTemp = (*frontptr);
    //check if the queue has only 1 item
    if((*frontptr)->next == nullptr && (*backptr)->next == nullptr){
        //means only one item is in the queue
        (*frontptr) = nullptr;
        (*backptr) = nullptr;
        delete deleteTemp;
        return;
    }
    (*frontptr) = (*frontptr) -> next;
    deleteTemp -> next = nullptr;

    //delete the node
    delete deleteTemp;
};
//displayall
template<typename T>
void display(const qNode<T>* frontptr) {

    if(frontptr==nullptr){
        std::cout << "The queue is empty." << std::endl;
        return;
    }
    const qNode<T>* temp = frontptr;
    while(temp != nullptr){
        std::cout << temp -> data;
```

```
        if(temp ->next != nullptr){
            std::cout << "->";
        }
        temp = temp ->next;
    }
    std::cout << std::endl;
};
//isempty
template <typename T>
bool isEmpty(qNode<T>** frontPtr, qNode<T>** backPtr) {
   return ((*frontPtr) == nullptr && (*backPtr) == nullptr);
}
#endif
```

Output

```
Front: H Back: H
Front: H Back: A
Front: H Back: R
Testing the dequeue operator
Front: A Back: R
Front: R Back: R
Testing the display operator
R
R->H
R->H->A->R
Is Empty? No
```

main.cpp

```cpp
#include <iostream>
#include "queuearr_ll.h"

int main(){
    // creating an object
    queueArr<int> q(5);

    // enqueue 10, 20, 30, 40, 50:
    q.enqueue(1);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(2);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(3);

    q.enqueue(4);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(5);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(6);

    std::cout << "\nremoved: " << q.dequeue() << std::endl;
    std::cout << "removed: " << q.dequeue() << std::endl;

    // testing the clear
    q.Clear();

    std::cout << "is the queue empty? "
              << (q.Empty() ? "yes" : "no")
              << std::endl;

    return 0;
}
```

queuearr_ll.h

```h
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>

// Array-Based Circular Queue
template <typename T>
class queueArr {
    private:
        // POINTER TO DYNAMICALLY ALLOCATE ARRAY
        T* q_arr;
        // MAXIMUM NUMBER OF ELEMENTS A QUEUE CAN HOLD
        size_t q_capacity;
        // CURRENT NUMBER OF ELEMENTS IN THE QUEUE
        size_t q_size;
        // INDEX OF THE FRONT ELEMENT IN THE QUEUE
        int q_front;
        // INDEX OF THE BACK ELEMENT IN THE QUEUE
        int q_back;

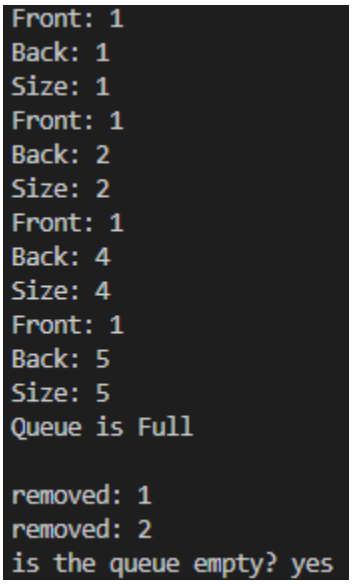
    public:
    // CONSTRUCTOR
    queueArr(size_t capacity) : q_capacity(capacity), q_size(0), q_front(0), q_back(-1) {
        // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
        q_arr = new T[q_capacity];
    }
    // COPY CONSTRUCTOR
    queueArr(const queueArr& other) : q_capacity(other.q_capacity), q_size(other.q_size), q_front(other.q_front), q_back(other.q_back) {
        // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
        q_arr = new T[q_capacity];
        // COPY ELEMENTS FROM OTHER QUEUE
        for (size_t i = 0; i < q_size; ++i) {
            q_arr[(q_front + i) % q_capacity] = other.q_arr[(other.q_front + i) % other.q_capacity];
        }
    }
    // COPY ASSIGNMENT OPERATOR
    queueArr& operator=(const queueArr& other) {
        if (this != &other) {
            // DEALLOCATE CURRENT ARRAY
            delete[] q_arr;
```

```
            // COPY CAPACITY AND SIZE
            q_capacity = other.q_capacity;
            q_size = other.q_size;
            q_front = other.q_front;
            q_back = other.q_back;
            // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
            q_arr = new T[q_capacity];
            // COPY ELEMENTS FROM OTHER QUEUE
            for (size_t i = 0; i < q_size; ++i) {
                q_arr[(q_front + i) % q_capacity] = other.q_arr[(other.q_front + i) % other.q_capacity];
            }
        }
        return *this;
    }
    // DESTRUCTOR
    ~queueArr() {
        // DEALLOCATE MEMORY FOR THE QUEUE ARRAY
        delete[] q_arr;
    }
    // QUEUE OPERATIONS
    bool Empty();
    bool Full();
    size_t Size();
    void Clear();
    T Front();
    T Back();
    void enqueue(T value);
    T dequeue();
};
// Empty()
template <typename T>
bool queueArr<T>::Empty() {
    // CHECK IF EMPTY
    return q_size == 0;
}
// Full()
template <typename T>
bool queueArr<T>::Full() {
    // CHECK IF REACHES MAX CAPACITY
    return q_size == q_capacity;
```

```cpp
// Size()
template <typename T>
size_t queueArr<T>::Size() {
    // RETURN CURRENT SIZE OF QUEUE
    return q_size;
}
// Front()
template <typename T>
T queueArr<T>::Front() {
    // Check if the queue is empty
    if (Empty()) {
        std::cout << "Queue is Empty\n";
        return T(); // Return default value of type T
    }
    // Return the front
    return q_arr[q_front];
}
// Back()
template <typename T>
T queueArr<T>::Back() {
    // Check if the queue is empty
    if (Empty()) {
        std::cout << "Queue is Empty\n";
        return T(); // Return default value of type T
    }
    // Return the back
    return q_arr[q_back];
}
// Clear()
template <typename T>
void queueArr<T>::Clear() {
    // Reset the queue to its initial state
    q_size = 0;
    q_front = 0;
    q_back = -1; // Set back to -1 since there are no elements
}
// Enqueue()
template <typename T>
void queueArr<T>::enqueue(T value) {
    // Check if the queue is full
    if (Full()) {
        std::cout << "Queue is Full\n";
        return;
    }
```

```
    // Increment the back
    q_back = (q_back + 1) % q_capacity;
    // Add the new value to the back of the queue
    q_arr[q_back] = value;
    // Increment the size of the queue
    q_size++;
}
// Dequeue()
template <typename T>
T queueArr<T>::dequeue() {
    // Check if the queue is empty
    if (Empty()) {
        std::cout << "Queue is Empty\n";
        return T(); // Return default value of type T
    }
    // Store the front value
    T value = q_arr[q_front];
    // Increment the front index in a circular manner
    q_front = (q_front + 1) % q_capacity;
    // Decrement the size of the queue
    q_size--;
    // Reset the indexes
    if (Empty()) Clear();
    // Return the dequeued value
    return value;
}

#endif // QUEUEARR_H
```

| | |
|---|---|
| Output |  |
| Observations | This is a basic implementation of queues in linked lists. By using templates, we are using pointers and addresses to traverse through the data. And apply the concepts of queues in the code blocks. Same thing with arrays which are a bit more complex for me since it uses a lot of commands and functions as well as more complex lines of codes. |

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

| ILO C : Solve Problems Using Queue Implementations | |
|---|---|
| Printerqueue.cpp (main) | |

```cpp
#include <iostream>
#include "queueing_ll.h"

int main(){
    Printer officePrinter;

    //Adding jobs
    officePrinter.addjob(101, "Alice", 15);
    officePrinter.addjob(102, "Bob", 3);
    officePrinter.addjob(103, "Charlie", 22);

    //Display queue
    officePrinter.displayQueue();

    //Processing job
    std::cout << "\nProcessing current job...\n";
    officePrinter.processNextJob();

    //Display queue after job
    officePrinter.displayQueue();

    //Process remaining jobs
    officePrinter.processAllJobs();

    return 0;
}
```

Observations: The main file only inputs or enqueues a job setup and it automatically accesses the header files where I've set everything up. All the main file is doing is accessing the commands and calling the function setup in the header files and displaying their results.

queueing _ll.h

```h
#ifndef queueing_ll_h
#define queueuing_ll_h
#include <iostream>
#include <string>

// job class
class job {
public:
    std::string username;
    int NumofPages;
    int id;

// class members and initializers
    job() {
        id = 0;
        username = "";
        NumofPages = 0;
    }

    job(int jobID, std::string name, int pages) {
        id = jobID;
        username = name;
        NumofPages = pages;
    }
};

// create a class node
template <typename T>
class qNode {
public:
    T data;
    qNode* next;
};

// new node creator
template<typename T>
qNode<T>* new_node(T newData) {
    // allocate space for the new node
    qNode<T>* newNode = new qNode<T>;
    // store the newData to the newNode
    newNode->data = newData;
    // point newNode next to null
    newNode->next = nullptr;
    return newNode;
}
```

```cpp
// enqueue
template<typename T>
void enqueue(T newData, qNode<T>** frontptr, qNode<T>** backptr) {
    // create a new node
    qNode<T>* newNode = new_node(newData);
    // check if the queue is empty
    if ((*frontptr) == nullptr && (*backptr) == nullptr) {
        (*frontptr) = newNode;
        (*backptr) = newNode;
        return;
    }
    // if not then insert at the back
    (*backptr)->next = newNode;
    (*backptr) = newNode;
}

// dequeue
template<typename T>
void dequeue(qNode<T>** frontptr, qNode<T>** backptr) {
    // check if the queue is empty
    if ((*frontptr) == nullptr && (*backptr) == nullptr) {
        std::cout << "The queue is empty." << std::endl;
        return;
    }
    // create a temporary variable to store the node to be deleted
    qNode<T>* deleteTemp = nullptr;
    // assign the current front to deleteTemp
    deleteTemp = (*frontptr);
    // check if the queue has only 1 item
    if ((*frontptr)->next == nullptr && (*backptr)->next == nullptr) {
        // means only one item is in the queue
        (*frontptr) = nullptr;
        (*backptr) = nullptr;
        delete deleteTemp;
        return;
    }
    (*frontptr) = (*frontptr)->next;
    deleteTemp->next = nullptr;
    // delete the node
    delete deleteTemp;
}
```

```cpp
// displayall
template<typename T>
void display(const qNode<T>* frontptr) {
    if (frontptr == nullptr) {
        std::cout << "The queue is empty." << std::endl;
        return;
    }
    const qNode<T>* temp = frontptr;
    while (temp != nullptr) {
        std::cout << temp->data;
        if (temp->next != nullptr) {
            std::cout << "";
        }
        temp = temp->next;
    }
    std::cout << std::endl;
}

// isempty
template <typename T>
bool isEmpty(qNode<T>** frontPtr, qNode<T>** backPtr) {
    return ((*frontPtr) == nullptr && (*backPtr) == nullptr);
}
// Printer class managing job queue
class Printer {
private:
    qNode<job>* front;
    qNode<job>* back;
public:
    Printer() {
        front = nullptr;
        back = nullptr;
    }
    void addjob(int id, std::string user, int pages) {
        job newjob(id, user, pages);
        enqueue(newjob, &front, &back);
    }
    void processNextJob() {
        if (isEmpty(&front, &back)) {
            std::cout << "No pending jobs to process.\n";
            return;
        }
        std::cout << "Processing: " << front->data << std::endl;
        dequeue(&front, &back);
    }
```

```
    void processAllJobs() {
        std::cout << "\nProcessing All Pending Jobs...\n";
        while (!isEmpty(&front, &back)) {
            processNextJob();
        }
        std::cout << "All jobs processed.\n";
    }
    void displayQueue() {
        std::cout << "\nCurrent Queue: " << std::endl;
        display(front);
    }
    ~Printer() {
        while (!isEmpty(&front, &back)) {
            dequeue(&front, &back);
        }
    }
};

#endif
```

Observations: This header acts as the system controller, connecting the inputs to the main driver. In my case, I used everything from the previous ILOB in order to pull my program off. By doing so, it allowed me to save time and utilize what I have in order to create something new. Reusing our ILOB program also allowed me to further understand how queues work in linked lists.

OUTPUT

```
Current Queue:
[ID: 101, User: Alice, Pages: 15]
[ID: 102, User: Bob, Pages: 3]
[ID: 103, User: Charlie, Pages: 22]


Processing current job...
Processing: [ID: 101, User: Alice, Pages: 15]


Current Queue:
[ID: 102, User: Bob, Pages: 3]
[ID: 103, User: Charlie, Pages: 22]


Processing All Pending Jobs...
Processing: [ID: 102, User: Bob, Pages: 3]

Processing: [ID: 103, User: Charlie, Pages: 22]

All jobs processed.
```

**OBSERVATIONS**

The output is simply just showing simple outputs where as you can see, shows you the current queue, the one it is currently processing, the current cue after processing the previous one which repeats until it has processed everything. I think I can still improve it by adding user inputs for more realism and application but for now, as I am not yet well versed in coding, I think I did well with this one.

5. I think that there's a lot more advantage to using linked lists instead of arrays especially when I was doing the activity. As you can see in my coding, I just copied everything from my previous ILO and inputted some setups such as the classes and the functions that I will need while applying what we had in the previous activities. I think I wouldn't be able to pull that off if I was using arrays since I have to set up a lot more and make sure that the queue doesn't exceed its capacity. In linked lists, I wouldn't have to do that since as long as I have the memory, I can keep on allocating resources and addresses for new entries.

COMPUTER ENGINEERING

QUEZON CITY

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** _____

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.